

DOGEstonia — Module 2 Canvas: Web2 Core / Story Intelligence Layer

Functional Requirements + продуктовая аналитика для разработки модуля

Статус: рабочий draft для постановки разработки.

Граница документа: только бизнес- и функциональный слой Web2 Core / Story Intelligence Layer; без инфраструктурных, API, SQL, векторных и DevOps-деталей.

Роль модуля в продукте: это операционное ядро между GPT-интервью и downstream-слоями. Продукт в целом нацелен на bottom-up сбор идей, желаний и жалоб жителей с их структурированием в цифровом контуре; доменная сущность витрины — **Issue**, а промежуточный web2-слой должен нормализовать данные в совместимый с SPA shape, не ломая UI.

1. Миссия модуля

Модуль Web2 Core / Story Intelligence Layer нужен для того, чтобы **не потерять живую историю человека**, но и **не завалить систему тысячами одинаковых жалоб**.

Он должен:

- принимать результат GPT-интервью как историю, а не как готовое бюрократическое обращение;
- сохранять историю как самостоятельную ценность;
- обогащать её смысловым профилем;
- позволять строить **динамические кластерные представления**;
- превращать созревшие кластеры в **distinct issues**;
- проецировать итог в **Issue**-формат, ожидаемый дашбордом;
- подготавливать доказательный пакет для будущей токенизации и/или внешней интеграции.

Проще: модуль превращает поток историй в **операционную память общества** и в **управляемые общественные кейсы**. Это соответствует общей логике DOGEstonia: ИИ собирает и структурирует, нода валидирует и нормализует, а дальше данные могут идти в дашборд, on-chain и во внешний контур.

2. Бизнес-задача, которую решает модуль

2.1. Исходная проблема

Если напрямую передавать в систему все истории жителей, возникают четыре перекоса:

1. **Шум и дубликаты**
Одно и то же системное нарушение приходит сотнями схожих формулировок.
2. **Потеря живой правды**
Если сразу схлопывать всё в тикеты, исчезает человеческий контекст.
3. **Ранняя фиксация смысла**
Если создавать жёсткие кластеры слишком рано, система “замораживает” реальность и теряет гибкость.
4. **Непригодность для Smart City / гос-контуров**
Внешняя система не может эффективно работать с массивом неструктурированных историй.

2.2. Целевое решение

Web2-модуль должен сделать возможной такую цепочку:

история → смысловой профиль → динамические кластерные выборки → distinct issue → дашборд / доказательный пакет

2.3. Продуктовый эффект

Если модуль работает правильно, то продукт получает:

- сохранённые индивидуальные истории;
 - возможность их многократной переагрегации;
 - distinct issues вместо потока дубликатов;
 - совместимость с текущим **Issue**-дашбордом;
 - трассируемую связь между публичным issue и первичными свидетельствами.
-

3. Score модуля

В score

Модуль обязан:

- принимать структурированный результат GPT-интервью;
- сохранять историю как первичную сущность;
- вести обогащённый смысловой профиль истории;
- поддерживать поиск и выборки по историям;
- строить динамические кластерные представления;
- выявлять кластеры, готовые к превращению в distinct issues;
- формировать **Issue**-проекции для SPA;

- готовить evidence pack для токенизации;
- сохранять происхождение каждого агрегированного кейса.

Out of scope

Модуль не должен:

- вести интервью с пользователем;
- принимать на себя роль policy/gate-движка как источника норм;
- токенизировать сам по себе;
- реализовывать on-chain голосование;
- быть источником финального гос-статуса;
- быть внешней Smart City интеграцией.

Пограничные зоны

Модуль **использует** результаты GPT и policy/gate, но не подменяет их.

Модуль **готовит** данные для Web3 и External Integration, но не выполняет их функции.

4. Основные бизнес-сущности модуля

4.1. Story

Первичная единица — отдельная история жителя.

Это не просто текст. История должна сохраняться как целостное гражданское свидетельство:

- что произошло;
- где произошло;
- кто/что затронуто;
- почему это важно;
- чего человек хочет вместо этого.

История — это не дубликат issue.

История — это минимальная единица общественного опыта.

4.2. Story Profile / Signal Profile

Смысловой профиль истории — это извлечённая надстройка над narrative.

Он нужен, чтобы история стала пригодной для:

- сопоставления с другими историями;
- динамической кластеризации;
- аналитики;
- будущей агрегации.

В него должны входить как минимум:

- поверхностная тема;
 - пространственный контекст;
 - объект городской среды;
 - тип системного сбоя;
 - глубинная потребность;
 - desired state;
 - общественная релевантность;
 - повторяемость.
-

4.3. Cluster View

Cluster View — это не вечный объект, а **вычисляемое или поддерживаемое представление** над набором историй.

Кластер нужен не ради таксономии, а ради ответа на вопрос:

“Какие истории сейчас разумно рассматривать вместе?”

Ключевой принцип:

- история может входить более чем в один кластер;
 - один и тот же набор историй может давать разные кластерные выборки под разные управленческие задачи.
-

4.4. Distinct Issue Candidate

Это кластер, который уже достаточно созрел, чтобы стать отдельным управляемым кейсом.

Он ещё не обязательно опубликован как итоговый issue, но уже:

- имеет устойчивое смысловое ядро;
- достаточно доказательств;
- имеет понятную географию;
- имеет общественную значимость;
- может быть выражен как единая проблема среды.

4.5. Distinct Issue

Distinct Issue — это уже **агрегированная общественная проблема**, пригодная для дашборда и для дальнейшей подачи во внешний контур.

Он не должен быть “средним арифметическим” по историям.
Он должен быть:

- кратким,
- управляемым,
- доказуемым,
- человеко-понятным,
- совместимым с **Issue**-контрактом SPA.

4.6. Evidence Pack

Evidence Pack — это доказательный пакет, привязанный к distinct issue или кластеру.

Он нужен, чтобы:

- сохранить связь агрегированного кейса с реальными историями;
- не перегружать основной issue-detail сырой массой данных;
- подготовить материал для токенизации, публичной верифицируемости и drill-down.

5. Философия модуля

5.1. Story-first, issue-later

Сначала сохраняется история, потом строится агрегированный смысл.

5.2. Не уничтожать первичное свидетельство

Агрегация не должна заменять историю.

Даже если создан distinct issue, история остаётся самостоятельным объектом доказательной базы.

5.3. Кластеры — это представления, а не догмы

Нельзя предполагать, что существует один “правильный” кластер.

Для продукта важнее поддержка разных аналитических и управленческих взглядов на один и тот же корпус историй.

5.4. Aggregation without dehumanization

Distinct issue должен убирать дублирование, но не лишать проблему человеческой плоти.

5.5. Traceability by design

Любой issue должен иметь обратный путь:

- к кластеру,
- к набору историй,
- к исходным narrative-свидетельствам.

5.6. Zero-change SPA как продуктивное ограничение

Web2-модуль должен работать так, чтобы текущий дашборд мог потреблять результат без перелома контракта: обязательные поля **Issue** уже определены, новые расширения не должны ломать BoardPage/IssuePage. Это прямо зафиксировано в проектных материалах и мосте GPT→SPA.

6. Бизнес-логика жизненного цикла

6.1. Приём истории

Модуль получает от GPT не готовое гос-обращение, а narrative package.

На этом шаге история должна быть признана как первичный объект и сохранена без принудительного схлопывания в issue.

6.2. Нормализация содержательного слоя

После приёма история должна стать пригодной для внутренней работы:

- понятна по теме;
- понятна по пространству;
- связана с глубинной потребностью;
- связана с желаемым состоянием.

6.3. Обогащение профилем

История дополняется смысловым профилем для дальнейших выборок и кластеризации.

6.4. Включение в кластерные представления

История становится доступной для:

- тематических,
- географических,
- системных,
- потребностных,
- временных и иных срезов.

6.5. Созревание кластера

Когда группа историй показывает устойчивое единство, она становится кандидатом в distinct issue.

6.6. Проекция в Issue

Distinct issue должен быть переведён в тот shape, который уже ожидает SPA: `id`, `status`, `type`, `labels`, `title`, `summary`, `description` и опционально `institution`, `created_at`, `arweave_txid`, `image_txid`, `image_hash`. При этом фиктивные txid запрещены.

6.7. Подготовка доказательного пакета

Отдельно от витрины issue должен формироваться evidence pack для будущей токенизации и/или внешней подачи.

7. Динамические кластеры: продуктовая модель

7.1. Почему кластеры должны быть динамическими

Фиксированные кластеры слишком рано “замораживают” смысл. В DOGEstonia история должна уметь быть прочитана под разными углами:

- как проблема парковки;
- как проблема предсказуемости жизни;
- как проблема конфликта использования пространства;
- как проблема конкретного района.

Поэтому кластер должен быть не навсегда закреплённой папкой, а управляемым представлением.

7.2. Основные оси кластеризации

Модуль должен поддерживать кластерные выборки как минимум по следующим осям:

- тема;
- география;
- тип пространства;
- тип системного сбоя;
- глубинная потребность;
- desired state;
- время / повторяемость;
- предполагаемая институциональная адресация.

7.3. Уровни кластеров

Модуль должен поддерживать по меньшей мере три масштаба:

- **микро-кейс** — локальная группа очень похожих историй;
- **локальный кластер** — проблема конкретного района/зоны;
- **системный кластер** — повторяющийся городской паттерн.

7.4. Множественная принадлежность

Одна история может одновременно:

- усиливать локальный кластер,
- подтверждать более широкий городской паттерн,
- входить в evidence pack для конкретного distinct issue.

Это обязательное свойство модели, а не исключение.

7.5. Кластер не равен issue

Не каждый кластер должен автоматически становиться distinct issue.

Часть кластеров нужна только для аналитики, наблюдения и выявления скрытых паттернов.

8. Distinct Issue: продуктовая логика

8.1. Что такое distinct issue

Distinct issue — это агрегированная формулировка общественной проблемы, которая:

- опирается на множество историй;
- не тонет в дублировании;
- сохраняет доказательную подложку;
- может быть показана на доске и подготовлена к дальнейшему движению.

8.2. Что distinct issue должен содержать по смыслу

Он должен отвечать не только на вопрос “что сломано”, но и на вопросы:

- где это проявляется;
- насколько это повторяемо;
- какой общественный вред создаёт;
- какая глубинная потребность жителей нарушена;
- какого улучшения люди ожидают.

8.3. Когда кластер готов стать issue

Для перехода к distinct issue нужно достаточное сочетание:

- смысловой связности;
- общественной повторяемости;
- географической или системной определённости;
- наличия осмысленного desired state;
- возможности кратко и честно сформулировать проблему без натяжки.

8.4. Что нельзя делать

Нельзя создавать distinct issue:

- только по одному ключевому слову;
- только по эмоции;
- только по количеству историй без смыслового ядра;
- если разные по природе проблемы ошибочно склеены.

9. Проекция в SPA-домен **Issue**

Текущий UI уже ожидает конкретный контракт. Обязательные поля:

- `id`
- `status`
- `type`

- `labels`
- `title`
- `summary` (опционально, иначе fallback на `title`)
- `description`

Опционально:

- `institution`
- `created_at`
- `arweave_txid`
- `image_txid`
- `image_hash`

Новые labels должны быть согласованы с фильтрами UI, а текстовые поля должны быть i18n-объектами `{ et, ru, en }`.

Для этого Web2-модуль должен быть не просто хранилищем историй, а **машиной проекций**:

- story layer,
- cluster layer,
- issue projection layer.

10. Functional Requirements

10.1. Роль и границы модуля

FR-M2-001. Модуль должен быть операционным источником истины по внутреннему жизненному циклу историй, кластерных представлений и issue-проекций до внешней интеграции.

FR-M2-002. Модуль должен принимать результат GPT-интервью как narrative package, а не как окончательно утверждённое обращение.

FR-M2-003. Модуль не должен считать GPT источником истины по статусам, публикации или внешнему принятию; эта логика в проекте принципиально вынесена из GPT.

FR-M2-004. Модуль должен отделять внутреннюю операционную правду от витринной проекции в `Issue`.

FR-M2-005. Модуль должен быть спроектирован так, чтобы поддерживать нативный домен `Issue`, а не старый `Activity`-домен.

10.2. Требования к приёму истории

FR-M2-006. Модуль должен сохранять каждую принятую историю как самостоятельный объект.

FR-M2-007. Модуль не должен принудительно схлопывать историю в already-существующий issue на шаге приёма.

FR-M2-008. Модуль должен сохранять исходный narrative как минимум в той форме, которая позволяет позднее перепроверить смысл агрегации.

FR-M2-009. Модуль должен принимать от GPT не только narrative, но и его структурированный смысловой слой, если он передан.

FR-M2-010. Модуль должен позволять хранить историю даже при отсутствии полной готовности к issue-проекции, если история ценна как свидетельство.

FR-M2-011. Модуль должен различать как минимум:

- полную историю,
 - частично заполненную историю,
 - историю, пригодную к кластеризации,
 - историю, пригодную к issue-проекции.
-

10.3. Требования к смысловому профилю истории

FR-M2-012. Для каждой истории модуль должен поддерживать смысловой профиль, достаточный для многомерной кластеризации.

FR-M2-013. Смысловой профиль должен отделять:

- тему;
- место;
- тип пространства;
- тип системного сбоя;
- глубинную потребность;
- desired state.

FR-M2-014. Модуль должен поддерживать возможность дообогащения истории без потери исходного narrative.

FR-M2-015. Модуль должен сохранять различие между тем, что сказал пользователь, и тем, что интерпретировано системой.

FR-M2-016. Модуль должен поддерживать хранение признаков повторяемости и общественной релевантности.

FR-M2-017. Если часть расширенного intake-слоя используется, модуль должен уметь хранить поля уровня `IssueIntakePayload` как внутренний operational layer до свёртки в публичный `Issue`. Такой слой уже рассматривается как кандидат для гос-логики и токенизации, но без навязывания PII по умолчанию.

FR-M2-018. Модуль не должен требовать PII как обязательного условия сохранения истории.

10.4. Требования к story store

FR-M2-019. Story store в Supabase должен быть первичным хранилищем живых историй, а не только кэшем под issue-карточки.

FR-M2-020. История должна сохраняться так, чтобы её можно было использовать:

- для аналитики,
- для кластеризации,
- для evidence pack,
- для последующего distinct issue.

FR-M2-021. Модуль должен сохранять происхождение истории: от какого интервью / intake она получена.

FR-M2-022. Модуль должен сохранять связь истории с её производными объектами, но без уничтожения первичного объекта.

FR-M2-023. Story store должен поддерживать поиск и выборки по смысловым признакам, а не только по дословному совпадению текста.

10.5. Требования к динамическим кластерным представлениям

FR-M2-024. Модуль должен поддерживать dynamic cluster views, а не только фиксированные кластеры.

FR-M2-025. Один и тот же корпус историй должен допускать более одного способа кластерного чтения.

FR-M2-026. Модуль должен поддерживать кластерные выборки как минимум по:

- теме,
- району / зоне,
- глубинной потребности,

- типу системного сбоя,
- desired state,
- времени.

FR-M2-027. Модуль должен позволять одной истории участвовать в нескольких cluster views.

FR-M2-028. Модуль должен поддерживать разные уровни масштаба кластеров: микро, локальный, системный.

FR-M2-029. Кластер не должен считаться “истинным навсегда”; модуль должен допускать пересборку кластерных представлений по новым критериям.

FR-M2-030. Модуль должен уметь показывать для кластера не только количество историй, но и доминирующее смысловое ядро.

FR-M2-031. Для каждого кластера модуль должен поддерживать краткое narrative-объяснение:

- о чём этот кластер;
- что объединяет входящие истории;
- в чём общественная цена проблемы.

FR-M2-032. Модуль должен различать аналитический кластер и issue-ready кластер.

FR-M2-033. Модуль не должен автоматически создавать distinct issue из каждого кластера.

10.6. Требования к формированию distinct issues

FR-M2-034. Модуль должен поддерживать явное состояние “cluster is issue-candidate”.

FR-M2-035. Distinct issue должен формироваться только при достаточной смысловой связности кластера.

FR-M2-036. При формировании distinct issue модуль обязан сохранить ссылочную связь с набором supporting stories.

FR-M2-037. Distinct issue должен содержать:

- краткое название проблемы;
- краткое резюме;
- полный narrative;
- предполагаемый type;
- согласованные labels;
- при наличии — institution;
- сведения о доказательной базе.

FR-M2-038. Distinct issue должен включать не только complaint framing, но и общественный смысл: какой неудовлетворённый городской паттерн он отражает.

FR-M2-039. Модуль должен уметь строить issue как для локальной проблемы района, так и для системного паттерна на уровне города.

FR-M2-040. Модуль не должен смешивать в один distinct issue истории с несовместимыми типами системного сбоя только из-за сходных слов.

FR-M2-041. Модуль должен позволять оператору/процессу разделять слишком широкий кластер на несколько issues.

FR-M2-042. Модуль должен позволять объединять несколько близких candidate issues, если выяснится, что они представляют один и тот же общественный паттерн.

10.7. Требования к проекции в SPA Issue

FR-M2-043. Модуль должен создавать issue-проекцию в виде, совместимом с текущим SPA-контрактом. Обязательные поля и ограничения уже зафиксированы в mock-layer guide и bridge-документе.

FR-M2-044. Для новой issue-проекции статус по умолчанию должен быть **NEW**, если иная бизнес-логика не будет отдельно утверждена. Такой статус соответствует текущему UI-словарю.

FR-M2-045. Модуль должен поддерживать **type** только из текущего рабочего словаря: **complaint, observation, absurdity, system_bug**.

FR-M2-046. **labels** в issue-проекции должны быть согласованы с фильтрами UI; новые labels не должны появляться как витринные без синхронизации со словарями доски.

FR-M2-047. **title, summary, description** должны формироваться как i18n-объекты **{ et, ru, en }**, поскольку именно такой формат ожидает текущая витрина.

FR-M2-048. Если **summary** отсутствует, issue-проекция должна поддерживать продуктовый fallback на **title**. Это соответствует текущему поведению карточки.

FR-M2-049. **institution, created_at, arweave_txid, image_txid, image_hash** должны рассматриваться как опциональные поля issue-проекции.

FR-M2-050. Модуль не должен подставлять фиктивные txid в issue-проекцию. Это прямо запрещено текущим guide.

FR-M2-051. Модуль должен поддерживать zero-change SPA как базовое ограничение MVP: любые расширения должны быть опциональными и не ломать существующее

потребление `issueService / BoardPage / IssuePage`. Этот принцип уже вынесен как отдельный вопрос и целевое допущение в `bridge`-документе.

10.8. Требования к `evidence pack` и подготовке к токенизации

FR-M2-052. Модуль должен уметь формировать `evidence pack` отдельно от публичной карточки `issue`.

FR-M2-053. `Evidence pack` должен сохранять ссылочную связь между `issue`, кластером и первичными историями.

FR-M2-054. `Evidence pack` должен быть пригоден для последующей токенизации `narrative/evidence`-слоя без изменения смысла первичных историй.

FR-M2-055. Модуль должен различать:

- что может быть показано публично;
- что остаётся во внутреннем `evidence layer`;
- что может стать `on-chain snapshot` позднее.

FR-M2-056. Модуль должен позволять готовить `evidence pack` как для отдельной истории, так и для кластера или `distinct issue`.

FR-M2-057. Модуль должен поддерживать неизбыточный принцип: `issue-card` не обязана показывать весь `evidence pack`, но должна быть связана с ним.

10.9. Требования к многоязычию и витринной читаемости

FR-M2-058. `Story` может храниться в исходном языке пользователя, но `issue`-проекция должна быть способна существовать в витринной `i18n`-форме.

FR-M2-059. Модуль должен различать “оригинальный `narrative`” и “витринный локализованный текст”.

FR-M2-060. При переводе в витринный слой модуль не должен выравнивать смысл до обезличенной канцелярщины.

FR-M2-061. Для `distinct issue` модуль должен формировать тексты так, чтобы они были одновременно:

- понятны человеку,

- пригодны для дашборда,
 - пригодны для дальнейшей гос-упаковки.
-

10.10. Требования к reviewability и человеческому контролю

FR-M2-062. Модуль должен позволять человеку или процессу review перепроверить, почему история попала в тот или иной кластер.

FR-M2-063. Модуль должен делать видимой логику issue-проекции: из каких историй и какого кластера она получена.

FR-M2-064. Модуль должен поддерживать сценарии split / merge / reframe для кластеров и issue-candidates.

FR-M2-065. Модуль должен позволять отложить issue-проекцию, если кластер ещё аналитически интересен, но недостаточно зрел для distinct issue.

10.11. Требования к privacy и минимизации данных

FR-M2-066. Модуль не должен требовать персонально идентифицирующие данные как базовое условие story intake.

FR-M2-067. Если расширенные intake-поля содержат потенциальный PII, модуль должен позволять держать их отдельно от публичной issue-проекции.

FR-M2-068. Публичный Issue должен быть производным объектом, не обязанным раскрывать внутренний story-layer целиком.

FR-M2-069. Модуль должен поддерживать идею “доказательная база без избыточной деанонимизации”.

FR-M2-070. История, кластер и issue должны быть функционально связаны, но не обязаны быть одинаково публичными.

11. Рабочая content-модель модуля

Ниже — не техническая схема, а бизнес-структура того, что модуль должен уметь держать.

11.1. Story Layer

- original narrative
- normalized narrative
- language
- location hint
- affected object
- emotional meaning
- deep need
- desired state
- public relevance
- repeatability signal

11.2. Story Intelligence Layer

- topic framing
- system-failure framing
- needs framing
- desired-state framing
- suggested institutional relevance
- cluster eligibility

11.3. Cluster Layer

- cluster lens
- cluster scope
- cluster narrative
- dominant patterns
- evidence volume
- issue-readiness

11.4. Distinct Issue Layer

- dashboard-ready title
- summary
- full description
- type
- labels
- institution
- creation timestamp
- evidence linkage

11.5. Evidence Layer

- story references

- supporting materials
 - cluster snapshot
 - issue snapshot
 - tokenization readiness marker
-

12. Что именно должно появиться в постановке разработки модуля

Для полноценной реализации Web2-модуля из этого canvas должны родиться как минимум следующие product specs:

Блок A. Story intake spec

Какой content-package считается принятой историей.

Блок B. Story intelligence spec

Какие смысловые признаки извлекаются и хранятся у истории.

Блок C. Cluster view spec

Какие виды кластерных представлений продукт поддерживает в MVP.

Блок D. Distinct issue promotion spec

По каким бизнес-правилам кластер становится issue-candidate и distinct issue.

Блок E. SPA projection spec

Как distinct issue превращается в текущий **Issue** shape без перелома UI. Это особенно критично, потому что продукт уже зафиксировал именно такую совместимость как целевую.

Блок F. Evidence pack spec

Как хранится и подготавливается доказательная связка story ↔ cluster ↔ issue.

13. Anti-patterns, которые модуль должен предотвращать

1. Схлопывание истории сразу в тикет
Потеря живого человеческого слоя.

2. **Плоская дедупликация по словам**
Разные по природе проблемы склеиваются только из-за общего словаря.
 3. **Жёсткие одноразовые кластеры**
Невозможность пересматривать общественную реальность под разными углами.
 4. **Issue без доказательной базы**
Карточка существует, но неясно, откуда она взялась.
 5. **Story swamp**
Истории хранятся, но не становятся управляемыми общественными кейсами.
 6. **Analytics without humanity**
Хорошая статистика, но полностью потерян narrative layer.
 7. **Narrative without governability**
Красивые истории, но невозможно подготовить distinct issue для Smart City.
 8. **Разрастание SPA-контракта**
Web2 пытается протащить в дашборд всё сразу и ломает текущую витрину.
-

14. Критерии качества модуля

Модуль высокого качества — это модуль, в котором:

- история не теряется;
 - кластеры не “цементируются” слишком рано;
 - distinct issues получаются ясными и честными;
 - каждую агрегированную карточку можно проследить назад к evidence base;
 - дашборд показывает управляемые кейсы, а не хаотичный поток сырого гражданского шума.
-

15. Acceptance Criteria

15.1. Story criteria

- История может быть сохранена без принудительного схлопывания в issue.
- У истории есть отделимый narrative и смысловой профиль.
- Историю можно использовать повторно в разных выборках.

15.2. Cluster criteria

- Один корпус историй может быть просмотрен через разные cluster views.
- Одна история может входить в более чем один кластер.
- Есть различие между аналитическим кластером и issue-ready кластером.

15.3. Distinct issue criteria

- Из зрелого кластера можно построить distinct issue.
- Distinct issue имеет обратимую связь к первичным историям.
- Distinct issue можно показать на дашборде в существующем **Issue**-контракте.

15.4. SPA criteria

- Проекция выдаёт **id, status, type, labels, title, summary, description**.
- **status** новой issue-проекции — **NEW**, если не задан иной процесс.
- **title / summary / description** имеют i18n-форму { **et, ru, en** }.
- Нет фиктивных txid.

15.5. Product criteria

- Модуль уменьшает поток дубликатов на уровне внешней подачи, не уничтожая доказательную базу.
 - Модуль позволяет строить distinct issues как collective cases, а не как одиночные тикеты.
 - Модуль создаёт мост между историями жителей и управляемыми кейсами продукта.
-

16. Рабочие допущения для текущей версии

1. Web2-модуль является центральным story-intelligence слоем продукта.
 2. Истории в Supabase — первичный внутренний актив, а не только материал для короткоживущей доски.
 3. Distinct issue — производный объект от story corpus, а не первичная запись.
 4. Поддерживается zero-change SPA как ограничение MVP.
 5. Расширенный intake-layer допустим, но без обязательного PII.
 6. Evidence rack и подготовка к токенизации входят в scope Web2 как подготовка, но не как сам Web3-процесс.
-

17. Открытые решения, которые повлияют на модуль

1. Какие cluster views обязательны уже в MVP, а какие можно отложить.
2. Должен ли каждый distinct issue иметь жёстко назначенную **institution**, или это может быть лишь suggested field.

3. Где проходит порог между “аналитически интересным кластером” и “issue-ready кластером”.
 4. Нужна ли отдельная витрина cluster cards до уровня official issue board.
 5. Какие именно расширенные поля intake-layer будут обязательны для evidence rack.
 6. Нужно ли поддерживать системные city-wide clusters уже в MVP или только районные.
 7. Нужна ли отдельная категория projection для “improvement case”, если operator later выделит её из **observation**.
-

18. Итоговая формула модуля

Web2 Core / Story Intelligence Layer в DOGEstonia — это **не просто backend-приёмник GPT**, а слой, который:

- сохраняет гражданские истории как первичные свидетельства,
- превращает их в смысловые сигналы,
- собирает из них динамические кластерные представления,
- формирует distinct issues для дашборда,
- и готовит доказательную подложку для токенизации и внешней интеграции,

при этом не ломая существующий **Issue**-контракт SPA и не сводя живой городской опыт к плоскому набору дубликатов.